UNIVERZITET U NIŠU

ELEKTRONSKI FAKULTET

SEMINARSKI RAD

Analiza sentimenta na nivou aspekata pomoću malih i velikih jezičkih modela

- OBRADA PRIRODNIH JEZIKA -

Mentor:
prof. dr Suzana Stojković

Student:
Jovan Paunović 2122

Niš, jun 2026. godina

SADRŽAJ

1. UVOD 3
2. ANALIZA SENTIMENTA 4
2.1. Pristupi analizi sentimenta 4
2.2. Vrste analize sentimenta 5
2.3. Analiza sentimenta na nivou aspekata 5
2.4. Izazovi analize sentimenta 6
3. JEZIČKI MODELI 7
3.1. *Transformer* arhitektura 7
3.2. Pretreniranje modela 8
3.3. Metode adaptacije pretreniranih modela 9
3.3.1. Metode zasnovane na upitu 9
3.3.2. Metode zasnovane na treniranju 10
3.4. Veliki i mali jezički modeli 11
3.4.1. Veliki jezički modeli 11
3.4.2. Mali jezički modeli 13
3.4.3. Poređenje velikih i malih jezičkih modela 13
4. EKSPERIMENTALNA EVALUACIJA 15
4.1. Odabrani modeli 15
4.1.1. Gemma 4 E2B 15
4.1.2. Qwen 3.5 2B 16
4.1.3. Gemini 3.5 Flash 16
4.2. Skup podataka 16
4.3. Postavka eksperimenta 17
4.4. Fino podešavanje 19
4.5. Evaluacione metrike 22
4.6. Rezultati i diskusija 24
5. ZAKLJUČAK 28
LITERATURA 29

1. UVOD

Analiza sentimenta predstavlja jedan od klasičnih problema obrade prirodnog jezika, čiji je cilj automatsko prepoznavanje stavova i emocija iz tekstualnih podataka. Primena ovakvih sistema je široka – od praćenja korisničkih recenzija do analize objava na društvenim mrežama – što je dovelo do intenzivnog istraživanja u ovoj oblasti tokom poslednjih dvadesetak godina.

Međutim, klasična analiza sentimenta svrstava ceo tekst u jednu kategoriju, što nije uvek dovoljno. Recenzija restorana može istovremeno sadržati pohvale na račun hrane i kritiku usluge, a oba stava se gube ako se tekst posmatra kao celina. Analiza sentimenta na nivou aspekata (engl. *Aspect Based Sentiment Analysis – ABSA*) rešava ovaj problem tako što zasebno određuje sentiment za svaki pomenuti aspekt, što je čini prikladnijom za praktične primene gde je potrebna finija informacija.

U ovom radu ispituje se koliko dobro mali i veliki jezički modeli rešavaju ABSA zadatak. Eksperimenti su sprovedeni na *SemEval 2014 Task 4* skupu podataka restoranskih recenzija, a evaluirani su *zero-shot*, *few-shot* i *fine-tuning* pristupi. Cilj je da se utvrdi da li manji modeli, koji su računarski znatno pristupačniji, mogu da pariraju većim modelima na ovom zadatku.

Seminarski rad je podeljen u četiri poglavlja. U poglavlju 2 dat je pregled analize sentimenta – osnovni pristupi, vrste i izazovi, sa posebnim osvrtom na analizu sentimenta na nivou aspekata. Poglavlje 3 pokriva jezičke modele, od *transformer* arhitekture i pretreniranja do poređenja malih i velikih modela i metoda adaptacije. Poglavlje 4 opisuje praktični deo rada, korišćeni skup podataka, evaluacione metrike i dobijene rezultate. Na kraju, poglavlje 5 iznosi zaključke i moguće pravce daljeg istraživanja.

2. ANALIZA SENTIMENTA

Analiza sentimenta predstavlja oblast obrade prirodnog jezika (engl. *Natural Language Processing – NLP*) koja se bavi automatskim prepoznavanjem i ekstrakcijom subjektivnih informacija iz teksta – stavova, mišljenja i emocija koje autori izražavaju prema određenim entitetima, proizvodima ili uslugama. Dok tradicionalne metode pretrage i analize teksta uglavnom rade sa objektivnim, činjeničnim sadržajem, analiza sentimenta fokusira se upravo na subjektivnu dimenziju jezika.

U savremenom digitalnom okruženju, korisnici svakodnevno ostavljaju ogromne količine tekstualnih podataka – recenzije proizvoda, komentare na društvenim mrežama, odgovore na ankete, poruke korisničke podrške. Ručna obrada podataka ovog obima nije izvodljiva, što čini automatizovanu analizu sentimenta izuzetno vrednom. Kompanije je koriste za praćenje reputacije brenda, razumevanje korisničkih iskustava, unapređenje proizvoda i donošenje poslovnih odluka zasnovanih na podacima [1], [2].

U ovom poglavlju opisuju se osnovni pristupi analizi sentimenta, nivoi na kojima se ona može vršiti, kao i analiza sentimenta na nivou aspekata koja predstavlja tehniku na kojoj se zasniva ovaj rad. Na kraju su istaknuti i glavni izazovi sa kojima se automatizovane metode susreću pri obradi prirodnog jezika.

2.1. Pristupi analizi sentimenta

Postoje tri osnovna pristupa analizi sentimenta: pristup zasnovan na pravilima, pristup zasnovan na mašinskom učenju i hibridni pristup [2].

Pristup zasnovan na pravilima oslanja se na unapred definisane rečnike reči sa pridruženim sentiment vrednostima – pozitivnim, negativnim ili neutralnim – uz skup lingvističkih pravila koja uzimaju u obzir modifikatore poput negacije ili pojačivača. Na primer, reč “loš” nosi negativnu vrednost, ali uz pojačivač “veoma” ta vrednost se pojačava, dok negacija “nije loš” menja polaritet. Prednost ovog pristupa je interpretabilnost i brzina, ali je njegova primena ograničena jer se teško nosi sa suptilnostima jezika, sarkazmom i novim izrazima koji nisu obuhvaćeni rečnikom.

Pristup zasnovan na mašinskom učenju koristi algoritme koji uče obrasce iz anotiranih skupova podataka. Klasični modeli poput *Naïve Bayes*-a, logističke regresije i SVM (engl. *Support Vector Machine*) dugo su bili dominantni u ovoj oblasti. Sa razvojem dubokog učenja, na scenu su stupile rekurentne neuronske mreže i *transformer* modeli, koji su sposobni da razumeju kontekst i semantiku na znatno višem nivou. Upravo u ovu kategoriju spadaju jezički modeli koji se koriste u ovom radu.

Hibridni pristup kombinuje oba navedena načina – lingvistička pravila obezbeđuju interpretabilnost i pouzdanost u jasnim slučajevima, dok modeli mašinskog učenja pokrivaju složenije i dvosmislene situacije. Ovaj pristup često daje najbolje rezultate, ali zahteva i najviše resursa za razvoj i održavanje.

2.2. Vrste analize sentimenta

Postoji nekoliko vrsta analize sentimenta koje se razlikuju po tome šta tačno mere i kakav izlaz proizvode [2].

Fino granulisana analiza sentimenta (engl. *fine-grained*) proširuje osnovnu klasifikaciju na 3 klase (pozitivno, negativno, neutralno) uvođenjem finijih kategorija – na primer, veoma pozitivno, pozitivno, neutralno, negativno i veoma negativno. Ovakav pristup omogućava preciznije merenje intenziteta sentimenta, slično načinu na koji sajtovi za recenzije koriste sistem ocenjivanja zvezdicama. Koristan je kada je važno razlikovati ne samo smer, već i jačinu izraženog stava.

Analiza sentimenta zasnovana na detekciji emocija prevazilazi koncept polariteta i pokušava da identifikuje konkretna emocionalna stanja iza teksta – sreću, tugu, ljutnju, iznenađenje, strah ili gađenje. Umesto da odredi samo da li je tekst negativan, ovaj pristup može prepoznati da li korisnik izražava frustraciju, razočaranje ili bes. Zbog složenosti i subjektivnosti emocija, ovo je tehnički najzahtevnija vrsta analize.

Analiza zasnovana na nameri (engl. *intent-based*) usmerena je na prepoznavanje korisnikove namere iza teksta – da li korisnik planira kupovinu, razmatra odustajanje od usluge, traži podršku ili izražava interesovanje za nadogradnju. Ova vrsta analize posebno je korisna u sistemima za korisničku podršku i *e-commerce* platformama.

Analiza sentimenta na nivou aspekata (ABSA) identifikuje konkretne karakteristike ili aspekte proizvoda i usluga koji se pominju u tekstu i za svaki od njih zasebno određuje sentiment. Za razliku od prethodno navedenih vrsta koje posmatraju tekst kao celinu, ABSA pruža granularni uvid u korisničko iskustvo – recenzija mobilnog telefona može istovremeno izražavati pozitivan stav prema kameri i negativan prema performansama. Upravo ova sposobnost čini ovu vrstu analize najkorisnijom vrstom analize za potrebe ovog rada, pa će biti detaljnije obrađena u narednoj sekciji.

2.3. Analiza sentimenta na nivou aspekata

Kao što je navedeno, ABSA identifikuje aspekte i za svaki određuje sentiment zasebno. Centralni pojam je upravo aspekt – karakteristika ili atribut entiteta o kome se govori u tekstu. U kontekstu recenzija restorana, aspekti mogu biti hrana, usluga, ambijent i slično. Identifikacija aspekata može se vršiti na različite načine: pristupom zasnovanim na pravilima i rečnicima, statističkim metodama ili, sve češće, primenom velikih jezičkih modela koji su sposobni da prepoznaju aspekte i u implicitnom obliku, bez njihovog eksplicitnog navođenja u tekstu [3].

ABSA zadatak se u literaturi najčešće formuliše kao kombinacija dva podzadatka. Prvi je ekstrakcija termina aspekata (engl. *Aspect Term Extraction – ATE*) – pronalaženje reči ili fraza u tekstu koje označavaju aspekte. Drugi je klasifikacija sentimenta aspekta (engl. *Aspect Term Sentiment Classification – ATSC*) – određivanje polariteta (pozitivan, negativan,

neutralan) za svaki identifikovani aspekt u datom kontekstu. U praksi se ova dva zadatka često spajaju u jedan, gde se aspekti i njihovi polariteti određuju istovremeno.

Sa pojavom pretreniranih *transformer* modela i velikih jezičkih modela, tačnost ABSA sistema značajno je porasla, tako da oni sada ostvaruju rezultate uporedive sa specijalizovanim modelima čak i bez dodatnog treniranja, a uz fino podešavanje postižu vrhunske performanse na standardnim *benchmark* skupovima podataka. Pored toga, detaljna formulacija upita (engl. *prompt engineering*) pokazala se kao ključan faktor za kvalitet rezultata u scenarijima bez dodatnog treniranja [4].

2.4. Izazovi analize sentimenta

Uprkos značajnom napretku u oblasti, automatizovana analiza sentimenta i dalje se suočava sa nizom izazova koji proizilaze iz prirodne složenosti ljudskog jezika [1].

Jedan od najčešćih problema je negacija – upotreba negativnih reči koje menjaju značenje iskaza. Rečenica poput “nisu me razočarali” zapravo izražava pozitivan stav, ali jednostavniji modeli zasnovani na rečnicima reč “razočarali” mogu pogrešno klasifikovati kao negativan signal. Slično važi i za pojačivače i umanjivače – “malo loš” i “izuzetno loš” nose isti polaritet, ali sasvim različit intenzitet.

Sarkazam i ironija predstavljaju još veći izazov, jer podrazumevaju upotrebu pozitivnih reči u negativnom kontekstu. Iskaz poput “Odlično, čekali smo obrok dva sata” sadrži reč pozitivnog značenja, ali izražava jasno nezadovoljstvo. Prepoznavanje ovakvog sadržaja zahteva razumevanje šireg konteksta, što je izvan dometa jednostavnih pristupa.

Problem konteksta javlja se i u situacijama gde isti izraz nosi različit sentiment u zavisnosti od postavljenog pitanja ili teme razgovora. Isti odgovor korisnika može biti pozitivan u jednom kontekstu i negativan u drugom, što sistem mora uzeti u obzir pri klasifikaciji.

Idiomatski izrazi i kolokvijalni jezik dodatno otežavaju analizu, jer njihovo doslovno tumačenje često ne odgovara stvarnom značenju. Ovaj problem posebno dolazi do izražaja u korisničkim recenzijama, koje su pisane neformalno i obiluju žargonom, skraćenicama i nestandardnim formulacijama.

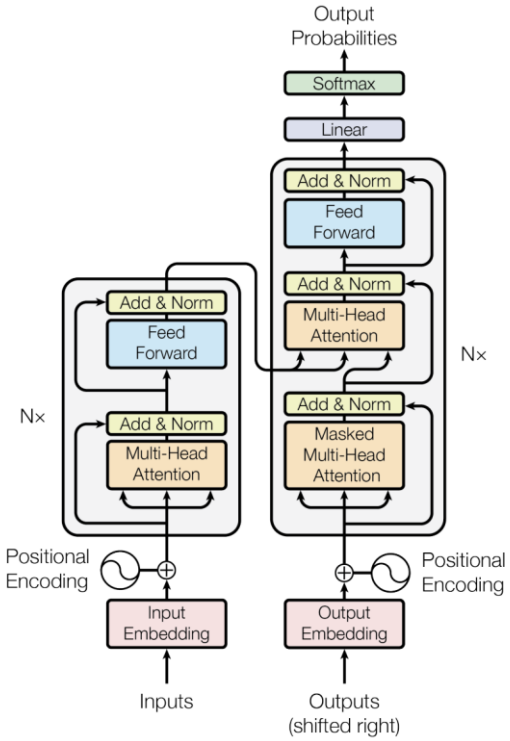
Upravo ovi izazovi opravdavaju primenu velikih i malih jezičkih modela u ove svrhe. Za razliku od pristupa zasnovanih na pravilima ili klasičnih modela mašinskog učenja, jezički modeli su sposobni da razumeju kontekst, prepoznaju implicitno značenje i nose se sa suptilnostima prirodnog jezika na način koji je znatno bliži ljudskom razumevanju teksta [1].

3. JEZIČKI MODELI

Jezički modeli zasnovani na *transformer* arhitekturi predstavljaju osnovu savremenih sistema za obradu prirodnog jezika. U ovom poglavlju dat je pregled ključnih koncepata potrebnih za razumevanje eksperimenata sprovedenih u radu – od arhitekture i načina treniranja, preko podele na male i velike modele, do metoda adaptacije za konkretne zadatke.

3.1. *Transformer* arhitektura

Pre pojave *transformer* arhitekture, dominantni pristupi obradi sekvencijalnih podataka u NLP-u bili su rekurentne neuronske mreže (RNN) i njihove varijante poput LSTM mreža. Osnovna mana ovih modela je serijska obrada ulaznog teksta – token po token, redom – što je otežavalo učenje zavisnosti između reči koje su daleko jedna od druge u rečenici. Pored toga, serijska priroda obrade onemogućavala je efikasnu paralelizaciju tokom treninga. *Transformer* arhitektura, predstavljena u radu [5], rešava oba problema uvođenjem mehanizma pažnje kao centralnog elementa. Arhitektura je prikazana na slici 3.1.



Slika 3.1. Transformer arhitektura [5]

Pre nego što tekst uđe u model, potrebno ga je prevesti u numerički oblik. Tekst se najpre deli na tokene – jedinice koje mogu biti reči, delovi reči ili znakovi interpunkcije – a svakom tokenu se dodeljuje odgovarajući vektor (engl. *embedding*) koji numerički predstavlja njegovo značenje [6]. Za razliku od RNN-ova koji implicitno čuvaju informaciju o redosledu

kroz serijsku obradu, *transformer* obrađuje sve tokene istovremeno. Zbog toga je neophodno eksplicitno dodati informaciju o poziciji svakog tokena u sekvenci, što se postiže pozicionim enkodiranjem – dodavanjem posebnog vektora koji kodira mesto tokena. Na ovaj način model zna ne samo šta je neki token, već i gde se nalazi u odnosu na ostale.

Ključni element *transformer* arhitekture je mehanizam pažnje (engl. *self-attention*) [5], [6]. Osnovna ideja je da model, prilikom obrade svakog tokena, uzima u obzir i ostale tokene u sekvenci i procenjuje koliko je svaki od njih relevantan u datom kontekstu. Na primer, u rečenici "Hrana je bila odlična, ali usluga užasna", reč "ali" signalizira da sentiment koji sledi nije isti kao onaj koji prethodi, što model treba da nauči da prepozna. Za svaki token se računaju tri vektora: *query* (šta ovaj token traži od konteksta), *key* (šta ovaj token može da ponudi) i *value* (informacija koju prenosi). Stepen pažnje između dva tokena određuje se poređenjem *query* vektora jednog i *key* vektora drugog tokena, a rezultat se koristi za težinsko sumiranje *value* vektora svih tokena u sekvenci. U praksi se koristi *multi-head attention*, gde se ovaj proces odvija paralelno u više "glava". Svaka glava uči da hvata drugačiji tip zavisnosti – neke glave se specijalizuju za sintaksičke odnose, druge za semantičke sličnosti, treće za koreferencijalne veze između reči. Izlazi svih glava se na kraju spajaju i transformišu u jedinstvenu reprezentaciju [6].

Originalna *transformer* arhitektura prikazana na slici 3.1. sastoji se od dva dela: enkodera i dekodera. Enkoder prima ulaznu sekvencu i kroz niz slojeva pažnje i potpuno povezanih slojeva gradi njenu kontekstualnu reprezentaciju. Dekoder koristi tu reprezentaciju kako bi, token po token, generisao izlaznu sekvencu – pri čemu svaki generisani token može da "gleda" sve prethodno generisane tokene, ali ne i buduće. Ovakav dizajn je prvobitno bio namenjen mašinskom prevođenju, gde enkoder čita izvorni jezik a dekoder generiše prevod. Kasniji modeli su preuzeli samo jedan od ova dva dela u zavisnosti od namene. BERT model koristi samo enkoder i pogodan je za zadatke razumevanja teksta poput klasifikacije i ekstrakcije informacija, dok GPT i njemu slični modeli koriste samo dekoder i namenjeni su generisanju teksta.

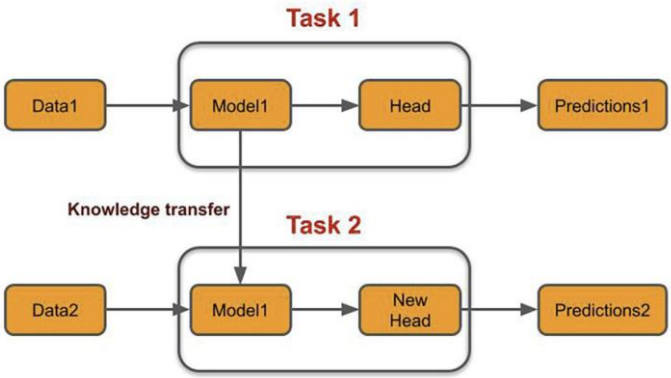
3.2. Pretreniranje modela

Transformer arhitektura sama po sebi definiše samo strukturu modela – ona ne nosi nikakvo znanje o jeziku. Da bi model postao koristan, potrebno ga je trenirati na velikoj količini tekstualnih podataka, i taj proces se naziva pretreniranje. Za razliku od klasičnih nadgledanih pristupa koji zahtevaju ručno označene primere, pretreniranje se zasniva na samo-nadgledanom učenju u kojem model sam iz sirovih podataka generiše signal za učenje, bez ikakve ljudske anotacije [7].

Konkretno, *decoder-only* modeli poput onih koji se koriste u ovom radu treniraju se kroz zadatak predviđanja sledećeg tokena. Model dobija sekvencu tokena i zadatak mu je da, na osnovu svih prethodnih, predvidi koji token dolazi sledeći. Ovaj naizgled jednostavan zadatak, ponavljan milijardama puta na raznovrsnm tekstovima, primorava model da nauči strukturu jezika, značenje reči i odnose između koncepata – sve bez ijednog ručno označenog

primera. *Encoder-only* modeli poput BERT-a koriste nešto drugačiji pristup – maskirano jezičko modelovanje (engl. *masked language modeling*), gde se nasumično sakriva deo tokena u rečenici a model pokušava da ih rekonstruiše koristeći kontekst sa obe strane.

Ključna pretpostavka pretreniranja je da znanje stečeno na jednom velikom skupu podataka može biti korisno za mnogo različitih zadataka. Ova ideja, poznata kao preneseno učenje (engl. *transfer learning*), drastično smanjuje potrebu za velikim skupovima označenih podataka za svaki konkretan zadatak [7]. Umesto da se model trenira od nule za svaki novi problem, pretrenirani model nosi generičku reprezentaciju jezika koja se dalje može iskoristiti ili prilagoditi, što je prikazano na slici 3.2. Upravo ova mogućnost adaptacije čini pretrenirane jezičke modele veoma praktičnim, a različite metode adaptacije detaljno su opisane u narednoj sekciji.



Slika 3.2. Preneseno učenje [8]

3.3. Metode adaptacije pretreniranih modela

Pretrenirani jezički model predstavlja polaznu tačku, ali sam po sebi nije nužno optimizovan za konkretan zadatak. Postoji nekoliko načina da se takav model prilagodi – od jednostavnog formulisanja upita, pa sve do dodatnog treniranja na domenski specifičnim podacima. Ove metode se razlikuju po tome koliko resursa zahtevaju i da li uopšte menjaju parametre modela [9].

3.3.1. Metode zasnovane na upitu

Zero-shot pristup podrazumeva da model dobija zadatak bez ijednog primera kako treba da izgleda izlaz. Model se oslanja isključivo na znanje stečeno tokom pretreniranja i pokušava da reši zadatak na osnovu instrukcije u upitu [10]. Na primer, za zadatak određivanja sentimenta aspekta upit može izgledati ovako:

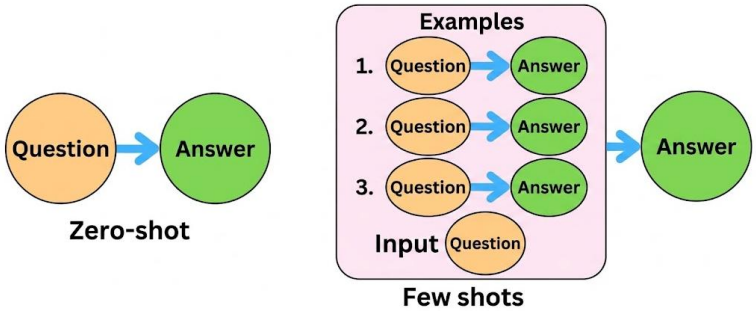
"Data je sledeća recenzija restorana, identifikuj aspekt i sentiment. Recenzija: Hrana je bila odlična, ali usluga je bila neverovatno spora."

Model treba da prepozna da se govori o dva aspekta – hrani (pozitivno) i usluzi (negativno) – bez ijednog prikazanog primera. Prednost ovog pristupa je što ne zahteva nikakve dodatne podatke ni resurse, međutim performanse mogu biti nepouzdane, posebno za specifičnije ili kompleksnije zadatke [9].

Few-shot pristup proširuje *zero-shot* tako što se u upit dodaje nekoliko primera ispravnih ulaz → izlaz parova. Model ne menja svoje težine – primeri služe kao kontekst koji usmerava generisanje odgovora [9]. Za isti ABSA zadatak, *few-shot* upit bi mogao da izgleda ovako:

„Data je sledeća recenzija restorana, identifikuj aspekte i sentimente za svaki pomenuti aspekt.
Recenzija: Testenina je bila ukusna, ali konobar je bio nepristojan. Aspekt: hrana, Sentiment: pozitivan. Aspekt: usluga, Sentiment: negativan.
Recenzija: Odlična atmosfera, ali pića su bila preskupa. Aspekt: atmosfera, Sentiment: pozitivan. Aspekt: cena, Sentiment: negativan.
Recenzija: Hrana je bila odlična, ali usluga je bila neverovatno spora."

Na ovaj način model dobija jasnu predstavu o očekivanom formatu i tipu zadatka. Istraživanja pokazuju da *few-shot* pristup može drastično poboljšati performanse u poređenju sa *zero-shot* – u nekim slučajevima i sa 19% na 97% tačnosti na istom zadatku [9]. Ključno je da primeri budu raznovrsni i reprezentativni za ceo skup podataka. Na slici 3.3. ilustrovana je razlika između *zero-shot* i *few-shot* pristupa.



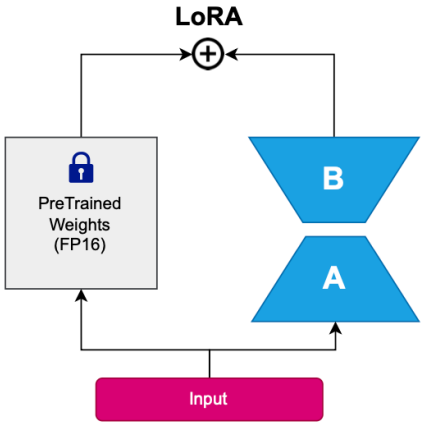
Slika 3.3. Razlika između zero-shot i few-shot pristupa

3.3.2. Metode zasnovane na treniranju

Fino podešavanje (engl. *fine-tuning*) podrazumeva dodatni trening pretreniranog modela na označenom skupu podataka specifičnom za zadatak, pri čemu se menjaju težine modela. Za razliku od *zero-shot* i *few-shot* pristupa koji ne utiču na parametre modela, fino podešavanje rezultuje novim modelom koji je specijalizovan za konkretan domen ili zadatak [11]. Nedostatak je što puno fino podešavanje gde se ažuriraju sve težine zahteva značajne računarske resurse i veliku količinu označenih podataka, što ga čini nepristupačnim bez odgovarajuće GPU infrastrukture.

Kao odgovor na ove probleme razvijene su metode efikasnog finog podešavanja (engl. *parameter-efficient fine-tuning – PEFT*), od kojih je najrasprostranjenija tehnika **LoRA** (engl. *Low-Rank Adaptation*). Osnovna ideja je da se originalne težine modela zamrznu i da se umesto

njih treniraju male matrice niskog ranga koje se ubacuju u određene slojeve [11]. Na ovaj način broj parametara koji se trenira drastično se smanjuje (u praksi i do nekoliko stotina puta) uz minimalan gubitak performansi u poređenju sa punim finim podešavanjem. Na slici 3.4. prikazan je princip rada LoRA metode – originalne težine modela su zamrznute, dok se ulaz paralelno propušta kroz dve matrice niskog ranga A i B čiji se izlaz sabira sa originalnim.



Slika 3.4. Princip rada LoRA metode [12]

Pored LoRA pristupa, postoji i **QLoRA** koja je proširenje LoRA metode koje kombinuje kvantizaciju modela sa LoRA adapterima. Model se učitava u smanjenoj numeričkoj preciznosti (npr. 4-bitna umesto standardne 16-bitne ili 32-bitne), što drastično smanjuje potrebnu GPU memoriju [11]. Zahvaljujući QLoRA, fino podešavanje modela sa milijardama parametara postaje izvodljivo čak i na običnim grafičkim karticama ili besplatnim *cloud* okruženjima poput *Google Colab*, što je bio slučaj i u ovom radu.

3.4. Veliki i mali jezički modeli

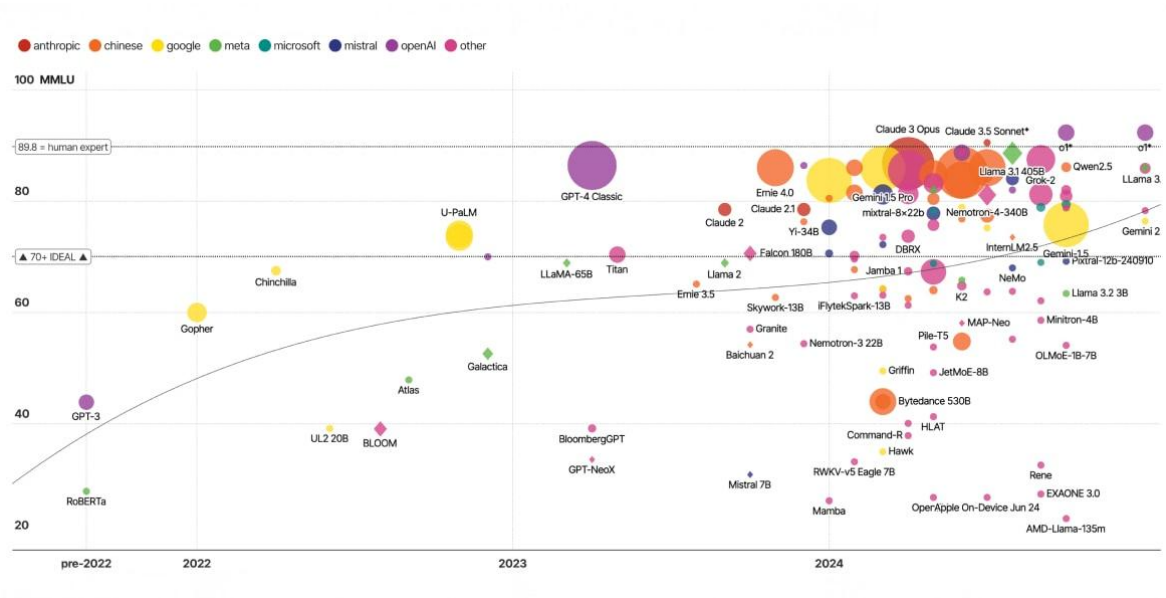
Razvoj jezičkih modela poslednjih godina obeležen je konstantnim povećanjem broja parametara, količine trening podataka i računarskih resursa potrebnih za treniranje. Ovaj trend skaliranja doveo je do prirodne podele na velike i male jezičke modele. Iako ne postoji formalno usvojena granica koja ih razdvaja, u literaturi se mali jezički modeli najčešće definišu kao modeli sa do 10 milijardi parametara, dok se modeli koji prevazilaze tu veličinu svrstavaju u kategoriju velikih jezičkih modela [13].

3.4.1. Veliki jezički modeli

Veliki jezički modeli (engl. *Large Language Models – LLM*) su modeli dubokog učenja trenirani na ogromnim količinama tekstualnih podataka, što ih čini sposobnim za razumevanje i generisanje prirodnog jezika na visokom nivou. Tipično sadrže stotine milijardi parametara i trenirani su na trilionima tokena iz raznovrsnih izvora – *web* stranica, knjiga, naučnih radova,

koda i drugog tekstualnog sadržaja [14]. Upravo ta raznovrsnost i obim podataka omogućavaju ovim modelima da razviju široko opšte znanje koje mogu primeniti na veliki broj različitih zadataka.

Razvoj LLM-ova usko je vezan za skaliranje: rast broja parametara i količine trening podataka. GPT-2 je 2019. privukao pažnju sposobnošću generisanja koherentnih paragrafa, dok je GPT-3 sa 175 milijardi parametara 2020. potvrdio potencijal ovog pristupa obradi prirodnog jezika [14]. Sa povećanjem veličine modela počinju da se pojavljuju sposobnosti koje manji modeli ne poseduju – poput rezonovanja kroz više koraka, rešavanja matematičkih zadataka i analognog zaključivanja – što je dodatno podstaklo istraživanja u ovoj oblasti. Ovaj trend rasta broja parametara kroz vreme prikazan je na slici 3.5.



Slika 3.5. Trend rasta broja parametara LLM-ova kroz vreme [15]

Zahvaljujući ovakvim sposobnostima, LLM-ovi se primenjuju u velikom broju scenarija. Koriste se za generisanje tekstualnog sadržaja poput mejlova, izveštaja i dokumentacije, kao i za sumarizaciju dugačkih dokumenata i naučnih radova. U domenu razvoja softvera, LLM-ovi pomažu programerima pri pisanju, pregledu i otklanjanju grešaka u kodu. Primenjuju se i za analizu sentimenta u korisničkim recenzijama i na društvenim mrežama, automatski prevod između jezika, kao i za izgradnju konverzacionih agenata koji mogu da vode složene dijaloge i izvršavaju zadatke u ime korisnika [14].

Međutim, LLM-ovi dolaze sa značajnim ograničenjima. Treniranje ovakvih modela je računarski izuzetno zahtevan proces – treniranje GPT-4 modela zahtevalo je 25.000 NVIDIA A100 grafičkih kartica koje su radile neprekidno tokom 90 do 100 dana [16]. Troškovi inferencije su takođe visoki, što znači da se većina LLM-ova ne može pokrenuti lokalno, već se koriste isključivo putem API-ja. Ovo otvara pitanje privatnosti podataka, jer se korisnički upiti šalju na eksterne servere, što u nekim industrijama poput finansija ili zdravstva može biti problematično. Pored toga, LLM-ovi su skloni halucinacijama – generisanju informacija koje zvuče uverljivo, ali su zapravo netačne – što zahteva pažljivu validaciju izlaza u kritičnim primenama [16].

3.4.2. Mali jezički modeli

Mali jezički modeli (engl. *Small Language Models – SLM*) su modeli za obradu prirodnog jezika koji su manji po obimu od LLM-ova. Broj parametara kreće se od nekoliko stotina miliona do nekoliko milijardi, što ih čini znatno kompaktnijim od svojih većih pandana [17]. Iako koriste istu *transformer* arhitekturu kao i LLM-ovi, njihova manja veličina uslovljava drugačiji način nastanka, drugačije prednosti i ograničenja.

SLM-ovi najčešće nastaju kompresijom većih modela primenom jedne ili više tehnika. Kao najčešće metode ističu se [17]:

- **Destilacija znanja** – prenos znanja sa većeg modela (modela učitelja) na manji model učenika, pri čemu se manji model trenira da oponaša ne samo izlaze, već i proces rezonovanja većeg modela,
- **Kvantizacija** – smanjenje preciznosti kojom se čuvaju težine modela, čime se značajno smanjuje memorijska zahtevnost bez drastičnog gubitka tačnosti,
- **Odsecanje** – uklanjanje manje važnih parametara iz mreže, uz naknadno fino podešavanje kako bi se nadoknadio eventualni gubitak tačnosti,
- **Faktorizacija niskog ranga** – dekomponovanje velike matrice težina u manje matrice nižeg ranga. Može biti računski zahtevna i takođe zahteva fino podešavanje kako bi se očuvala tačnost modela.

Kompaktnost SLM-ova donosi niz praktičnih prednosti. Mogu se pokretati lokalno, bez potrebe za skupom GPU infrastrukturom ili pristupom eksternim API servisima, što je posebno važno u scenarijima gde je privatnost podataka od ključnog značaja [16]. Niži broj parametara direktno se odražava na brzinu – SLM-ovi generišu odgovore brže i uz manje energetske troškove, što ih čini ekološki prihvatljivijim rešenjem. Važna prednost je i izvodljivost finog podešavanja – dok je puno fino podešavanje LLM-ova praktično nedostupno bez ozbiljne infrastrukture, SLM-ovi se mogu efikasno prilagoditi konkretnom zadatku metodama poput QLoRA čak i na potrošačkim grafičkim karticama.

Ipak, SLM-ovi imaju i svoja ograničenja. Zbog manjeg kapaciteta, postižu slabije rezultate na kompleksnim zadacima koji zahtevaju široko opšte znanje ili duboko kontekstualno razumevanje. Ograničenija generalizacija znači da su pogodniji za ciljane, domenski specifične primene nego za zadatke opšte namene. Kao i LLM-ovi, skloni su halucinacijama, a manji kapacitet može dodatno pojačati ovaj problem na zadacima van domena za koji su optimizovani. Primeri popularnih SLM-ova uključuju modele poput *Gemma*, *Qwen*, *Phi*, *DistilBERT*, *Granite*, *Ministral* i *Llama 3.2*.

3.4.3. Poređenje velikih i malih jezičkih modela

Iako obe kategorije modela dele istu osnovu arhitekture, razlikuju se u gotovo svakom praktičnom aspektu – od resursa potrebnih za treniranje i pokretanje, do toga za koje zadatke su prikladniji. Tabela 1. daje pregled ključnih razlika [18].

Tabela 1. *Poređenje karakteristika velikih i malih jezičkih modela*

Karakteristika	**Veliki jezički modeli (LLM)**	**Mali jezički modeli (SLM)**
Broj parametara	Milijarde do trilioni	Stotine miliona do nekoliko milijardi
Infrastruktura	Zahteva moćne GPU klastere, *cloud deployment*	Može da radi lokalno, na mobilnim uređajima
Latencija	Viša	Niža, pogodni za primene u realnom vremenu
Trošak treniranja i korišćenja	Veoma visok	Znatno niži
Privatnost podataka	Upiti se šalju na eksterne servere	Lokalno pokretanje, bolji nivo privatnosti
Fino podešavanje	Računarski zahtevno, teško izvodljivo	Izvodljivo čak i na potrošačkom hardveru
Generalizacija	Široka, radi dobro na raznovrsnim zadacima	Ograničenija, bolja za specifične domene

4. EKSPERIMENTALNA EVALUACIJA

U ovom poglavlju opisani su eksperimenti sprovedeni u cilju evaluacije sposobnosti malih i velikih jezičkih modela na zadatku analize sentimenta na nivou aspekata. Najpre su predstavljeni odabrani modeli i njihove ključne karakteristike, nakon čega sledi opis korišćenog skupa podataka i pripreme eksperimenta. Detaljno su opisane metode evaluacije i postupak finog podešavanja, a rezultati su prikazani i diskutovani u završnim podsekcijama.

4.1. Odabrani modeli

Za potrebe ovog istraživanja odabrana su tri jezička modela koji pokrivaju različite slučajeve po pitanju veličine i načina korišćenja: dva mala, otvorena modela koja se pokreću lokalno – *Gemma 4 E2B* i *Qwen 3.5 2B* – i jedan veliki model dostupan putem API-ja – *Gemini 3.5 Flash*. Ovakav izbor omogućava direktno poređenje performansi SLM i LLM pristupa na istom zadatku i skupu podataka, uz uvid u to koliko resursa i infrastrukture svaki od njih zahteva.

4.1.1. Gemma 4 E2B

Gemma 4 E2B je mali otvoreni model razvijen od strane *Google DeepMind*, objavljen pod *Apache 2.0* licencom. Slovo "E" u nazivu označava "efektivne" parametre – model ima 2.3 milijarde efektivnih parametara, dok ukupan broj parametara uključujući *embedding* slojeve iznosi 5.1 milijardi. Ova razlika postoji zbog tehnike *Per-Layer Embeddings* (PLE), kojom svaki dekoder sloj dobija sopstveni mali *embedding* za svaki token, što povećava efikasnost na uređajima sa ograničenim resursima bez dodavanja novih slojeva [19].

Model koristi hibridni mehanizam pažnje koji kombinuje lokalni klizni prozor pažnje sa globalnom pažnjom, pri čemu je poslednji sloj uvek globalan. Kontekstni prozor iznosi 128 hiljada tokena. *Gemma 4 E2B* je multimodalan model koji podržava tekst, slike i audio kao ulaz. Kao i ostali modeli iz *Gemma 4* familije, E2B je dizajniran sa mogućnošću rezonovanja (engl. *thinking mode*) čiji se nivo može podešavati. U ovom radu korišćen je bez aktiviranog mehanizma rezonovanja. Model je posebno optimizovan za lokalno pokretanje na laptopovima i mobilnim uređajima, što ga čini pogodnim za scenarije gde nije dostupna serverska infrastruktura [19]. U ovom radu model je pokretan lokalno putem *Ollama* okruženja.

4.1.2. Qwen 3.5 2B

Qwen 3.5 2B je mali otvoreni model razvijen od strane *Alibaba* kompanije, takođe dostupan pod *Apache 2.0* licencom. Model ima 2 milijarde parametara i koristi hibridnu arhitekturu koja kombinuje *Gated Delta Network* slojeve sa standardnim slojevima pažnje, što obezbeđuje visok protok pri inferenciji uz minimalnu latenciju. Kontekstni prozor iznosi 262.144 tokena, što je znatno više od većine modela sličnih veličina.

Jedna od ključnih karakteristika *Qwen 3.5* serije je podrška za 201 jezik i dijalekt, što ga čini jednim od najšire primenjivanih malih modela sa aspekta višejezičnosti. Model je multimodalan i podržava tekst, slike i video kao ulaz. Pored toga, *Qwen 3.5 2B* poseduje ugrađeni mehanizam rezonovanja koji modelu omogućava da pre davanja odgovora interno prođe kroz korake zaključivanja, što može poboljšati performanse na složenijim zadacima [20]. U ovom radu model je pokretan lokalno putem *Ollama* okruženja, u standardnom režimu bez aktiviranog mehanizma rezonovanja.

4.1.3. Gemini 3.5 Flash

Gemini 3.5 Flash je veliki jezički model razvijen od strane *Google DeepMind*, predstavljen u maju 2026. godine. Za razliku od prethodna dva modela, *Gemini 3.5 Flash* nije dostupan kao otvoreni model već isključivo putem API-ja, što ga svrstava u kategoriju zatvorenih komercijalnih modela. Prema zvaničnoj dokumentaciji, model postiže vrhunski nivo performansi na više zadataka, nadmašujući *Gemini 3.1 Pro* na zahtevnim benčmarkovima za kodiranje i agentičke zadatke, uz brzinu generisanja koja je četiri puta veća od ostalih najnovijih modela. Model je posebno optimizovan za složene agentičke zadatke i postiže visoke rezultate na multimodalnim benčmarkovima [21].

U kontekstu ovog rada, *Gemini 3.5 Flash* služi kao predstavnik LLM kategorije i referentna tačka za poređenje sa lokalnim SLM modelima. Korišćen je putem zvaničnog *Gemini* API-ja.

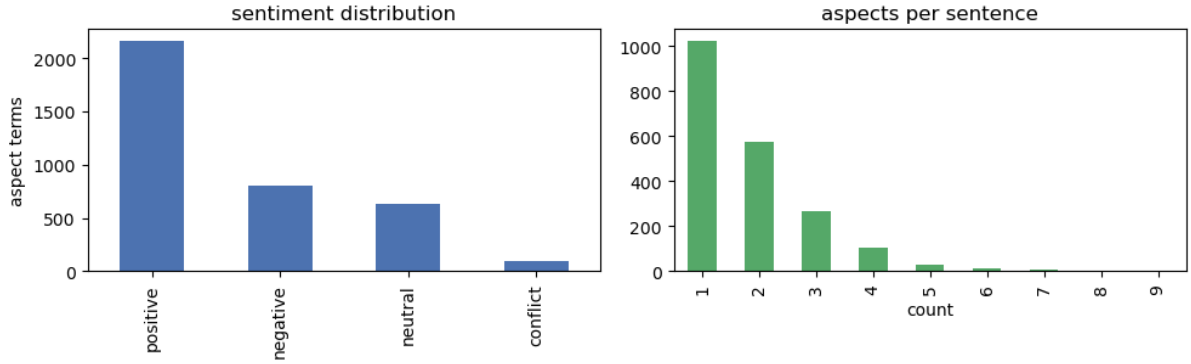
4.2. Skup podataka

U ovom radu korišćen je *SemEval 2014 Task 4* skup podataka [22], koji predstavlja jedan od najčešće korišćenih referentnih skupova podataka u oblasti analize sentimenta na nivou aspekata. Skup podataka sadrži recenzije restorana na engleskom jeziku, ručno anotiran parovima (aspekt, sentiment), gde svaki aspekt predstavlja reč ili frazu preuzetu iz teksta recenzije, a sentiment je jedna od četiri klase: pozitivan, negativan, neutralan ili konfliktni (istovremeno pozitivan i negativan).

Originalni skup podataka dolazi u dugom formatu (jedna vrsta po aspektu) što znači da jedna rečenica može biti zastupljena u više redova ako sadrži više aspekata. Za potrebe ovog rada, podaci su rekonstruisani na nivo rečenice, gde je svaki primer rečenica zajedno sa listom

svih njenih (aspekt, sentiment) parova. Nakon rekonstrukcije, skup podataka sadrži ukupno 2021 rečenicu i 3693 aspekta, što daje prosek od 1.83 aspekta po rečenici, sa maksimumom od 9 aspekata u jednoj rečenici. Prosečna dužina rečenice iznosi 82 karaktera. Važno je napomenuti da nijedna rečenica nije bez ijednog aspekta, što znači da su sve rečenice u skupu podataka relevantne za zadatak.

Raspodela sentimenta je neravnomerna, što je karakteristično za ovakve skupove podataka. Dominira pozitivan sentiment sa 2164 pojavljivanja (58.6%), sledi negativan sa 805 (21.8%), neutralan sa 633 (17.1%), dok je konfliktni sentiment najređi sa svega 91 primerom (2.5%). Ovakva neravnomerna raspodela direktno utiče na izbor metrika evaluacije – tačnost sama po sebi nije dovoljan pokazatelj kvaliteta modela jer model koji uvek predviđa pozitivan sentiment može postići visoku tačnost, a da zapravo ne razume zadatak. Zbog toga je kao dopunska metrika korišćen makro F1, koji jednako vrednuje sve klase bez obzira na njihovu zastupljenost. Na slici 4.1. prikazana je raspodela sentimenata po aspektima u skupu podataka (levo), kao i distribucija broja aspekata po rečenici (desno).



Slika 4.1. Raspodela sentimenata po aspektima u skupu podataka (levo) i distribucija broja aspekata po rečenici (desno)

Skup podataka je nasumično podeljen na trening i test skup u odnosu 80:20, pri čemu je podela izvršena na nivou rečenica i sa fiksiranim *seed*-om kako bi rezultati uvek bili isti pri svakom pokretanju. Trening skup sadrži 1616 rečenica, a test skup 405 rečenica. Isti test skup korišćen je za evaluaciju svih modela i svih eksperimentalnih režima – *zero-shot*, *few-shot* i *fine-tuning* – čime je osigurana uporedivost rezultata.

4.3. Postavka eksperimenta

Lokalna inferencija sprovedena je na laptopu sa *AMD Ryzen AI 5 340* procesorom (6 jezgara, 12 logičkih procesora, bazna frekvencija 2.00 GHz) i 16 GB RAM memorije. Grafička kartica je *AMD Radeon 840M* sa svega 412 MB VRAM memorije, što je daleko ispod minimuma potrebnog za pokretanje jezičkih modela na GPU-u. Zbog toga su lokalni modeli u *zero-shot* i *few-shot* režimu pokrenuti na procesoru. Fino podešavanje, kao i evaluacija fino podešenih modela, sprovedeni su odvojeno u *Google Colab* okruženju radi pristupa GPU resursima. Rezultujuće predikcije su zatim učitane u glavnu svesku zajedno sa predikcijama

lokalnih modela, kako bi se metrike za sve modele i režime računale na jedinstven način. Detalji *fine-tuning* postupka opisani su u narednoj sekciji.

Za pokretanje lokalnih modela korišćeno je *Ollama* okruženje – alat koji omogućava preuzimanje i pokretanje jezičkih modela u GGUF formatu direktno na CPU-u bez potrebe za posebnom konfiguracijom. *Gemma 4 E2B* pokrenuta je kao *gemma4:e2b*, a *Qwen 3.5 2B* kao *qwen3.5:2b-q4_K_M*. Oba modela učitana su u Q4_K_M kvantizaciji – 4-bitnom GGUF formatu koji je razvijen u okviru *llama.cpp* projekta i selektivno primenjuje nižu preciznost na manje važne slojeve modela, čime se postiže dobar balans između veličine modela i kvaliteta generisanja.

Kako bi merenja memorije i vremena izvršavanja bila pripisiva konkretnom modelu, model je posle svakog eksperimentalnog režima eksplicitno uklonjen iz memorije pre učitavanja sledećeg. Svaki model je pokrenut jednom i korišćen za oba režima (*zero-shot* i *few-shot*), što je osiguralo konzistentnost uslova merenja. Temperatura generisanja postavljena je na 0 radi determinisanosti.

Gemini 3.5 Flash korišćen je putem zvaničnog *Google Gemini* API-ja. Budući da se inferencija odvija na *Google*-ovoj infrastrukturi, merenje memorije nije bilo moguće – beleži se samo vreme odziva koje uključuje i mrežnu latenciju. Temperatura je postavljena na 0, a *thinking mode* na minimalni nivo jer API ne dozvoljava potpuno isključivanje. Eventualni tragovi rezonovanja uklanjaju se iz izlaza pre evaluacije kako bi rezultati bili uporedivi sa lokalnim modelima.

Svi modeli koristili su identičan format upita. Upit se sastoji od sistemske instrukcije koja definiše zadatak i nalaže modelu da vrati isključivo JSON niz oblika [{"*aspect*": "<*term*>", "*sentiment*": "<*sentiment*>"}], bez ikakvih dodatnih objašnjenja. Dozvoljene vrednosti sentimenta su četiri: *positive*, *negative*, *neutral* i *conflict*, a za rečenice bez aspekata model treba da vrati prazan niz. U *zero-shot* režimu upit se sastoji samo iz sistemske instrukcije sa rečenicom koja se analizira. U *few-shot* režimu upitu su dodati primeri iz trening skupa (po jedan za svaku od četiri klase sentimenta) prikazani kao parovi rečenica i očekivanog JSON izlaza. Sistemska instrukcija i postupak formiranja upita prikazani su u Listingu 1, pri čemu se ista *build_prompt* funkcija koristi za oba režima, pri čemu se u *few-shot* režimu prosleđuju i primeri.

Listing 1. Sistemska instrukcija i formiranje upita

```
SYSTEM_INSTRUCTION = (
    "You are an aspect-based sentiment analysis system for customer reviews.\n"
    "Given one sentence, extract every aspect term that is explicitly mentioned and its
sentiment.\n"
    "An aspect term is a word or phrase copied verbatim from the sentence that names something
being evaluated.\n"
    "sentiment is exactly one of: positive, negative, neutral, conflict (conflict = both
positive and negative).\n"
    "If the sentence contains no aspect term, return an empty array.\n"
    "Return ONLY a JSON array and nothing else. "
    'Each element must be {"aspect": "<term>", "sentiment": "<sentiment>"}.'
)
```

```
def build_prompt(text, examples=None):
    parts = [SYSTEM_INSTRUCTION, ""]
    if examples:
        for ex in examples:
            parts.append(f'Sentence: "{ex["text"]}"')
            parts.append("Output: " + json.dumps(
                [{"aspect": p["aspect"], "sentiment": p["sentiment"]} for p in ex["aspects"]],
                ensure_ascii=False))
            parts.append("")
    parts.append(f'Sentence: "{text}"')
    parts.append("Output:")
    return "\n".join(parts)
```

Parsiranje izlaza implementirano je kroz dva nivoa: najpre se pokušava direktno parsiranje JSON-a, a ako to ne uspe onda se pokušava ekstrakcija niza uz uklanjanje tragova rezonovanja, a ako oba koraka ne uspeju beleži se greška u formatu odgovora, što je prikazano u Listingu 2.

Listing 2. Parsiranje izlaza modela kroz dva nivoa

```
def _strip_think(text):
    return re.sub(r"<think>.*?</think>", "", text, flags=re.DOTALL | re.IGNORECASE)

def _extract_array(text):
    s, e = text.find("["), text.rfind("]")
    return text[s:e + 1] if (s != -1 and e > s) else None

def parse_prediction(raw):
    # vraća (pairs, ok); ok=False -> izlaz nije bio u traženom JSON formatu
    text = re.sub(r"```(?:json)?", "", _strip_think(str(raw))).strip()
    for cand in (text, _extract_array(text)):
        if not cand:
            continue
        try:
            data = json.loads(cand)
        except Exception:
            continue
        pairs = _normalize_pairs(data)
        if pairs is not None:
            return pairs, True
    return [], False
```

4.4. Fino podešavanje

Pored evaluacije u *zero-shot* i *few-shot* režimu, oba SLM modela dodatno su fino podešena na trening skupu kako bi se ispitalo u kojoj meri dodatno treniranje poboljšava

performanse na ABSA zadatku. Trening upit je identičan *zero-shot* upitu iz glavne sveske, čime se obezbeđuje da adapter uči da odgovori tačno na onaj format ulaza koji će dobijati prilikom evaluacije.

Za fino podešavanje korišćena je biblioteka *Unsloth*, koja optimizuje proces treniranja u pogledu brzine i memorijske potrošnje. Oba modela učitana su u 4-bitnoj NF4 kvantizaciji (QLoRA pristup) – originalne težine modela su zamrznute, dok se treniraju samo LoRA adapteri. Konfiguracija LoRA adaptera je ista za oba modela: *r=16, alpha=16, dropout=0, bias= "none"*. Učitavanje Gemma modela prikazano je u Listingu 3. *Qwen 3.5* se zbog svoje multimodalne arhitekture učitava preko *FastVisionModel*, pri čemu se *vision* slojevi zamrzavaju jer je zadatak isključivo tekstualni (Listing 4), dok je konfiguracija LoRA adaptera ista kao kod *Gemma* modela.

Listing 3. Učitavanje Gemma 4 E2B modela u 4-bitnoj kvantizaciji i dodavanje LoRA adaptera

```
model, tokenizer = FastModel.from_pretrained(
    model_name = "unsloth/gemma-4-E2B-it",
    max_seq_length = 1024,
    load_in_4bit = True,
    full_finetuning = False,
)

model = FastModel.get_peft_model(
    model,
    r = 16, lora_alpha = 16, lora_dropout = 0, bias = "none",
    target_modules = ["q_proj","k_proj","v_proj","o_proj",
                      "gate_proj","up_proj","down_proj"],
    use_gradient_checkpointing = "unsloth",
    random_state = 42,
)
```

Listing 4. Učitavanje Qwen 3.5 2B modela u 4-bitnoj kvantizaciji i dodavanje LoRA adaptera

```
model, tokenizer = FastVisionModel.from_pretrained(
    "unsloth/Qwen3.5-2B",
    load_in_4bit = True,
    use_gradient_checkpointing = "unsloth",
    max_seq_length = 1024,
)

model = FastVisionModel.get_peft_model(
    model,
    finetune_vision_layers     = False,
    finetune_language_layers   = True,
    finetune_attention_modules = True,
    finetune_mlp_modules       = True,
    r = 16, lora_alpha = 16, lora_dropout = 0, bias = "none",
    random_state = 42,
)
```

Trening primeri konstruisani su iz trening skupa u istom formatu kao *zero-shot* upit, sa očekivanim izlazom u obliku JSON niza. Kod *Gemma* modela format je jedinstven string oblika upit+odgovor, dok je kod *Qwen* modela korišćen *chat* format. Ova razlika u formatu kod Qwen modela prikazana je u Listingu 5. Rečenice bez aspekata mapirane su na praznu listu, čime se model uči da ne izmišlja aspekte kada ih nema.

Listing 5. Konstrukcija primera u chat formatu (Qwen)

```
def to_conversation(s):
    return {"messages": [
        {"role": "user",      "content": [{"type": "text", "text": build_prompt(s["text"])}]},
        {"role": "assistant", "content": [{"type": "text", "text": to_completion(s)}]},
    ]}
```

Oba modela trenirana su kroz 3 epohe, sa veličinom *batch*-a 2 i *gradient accumulation* korakom 4, stopom učenja 2e-4, 8-bitnim *AdamW* optimizatorom i linearnim opadanjem stope učenja. Korišćeno je 5% trening skupa kao validacioni skup radi praćenja gubitka tokom treninga. Tabele 2 i 3 prikazuju trening i validacioni gubitak po epohama za oba modela.

Tabela 2. *Gubitak na trening i validacionom skupu kod Gemma 4 E2B modela*

Epoha	Training Loss	Validation Loss
1	0.190784	0.788230
2	0.069666	0.702740
3	0.045449	0.690689

Tabela 3. *Gubitak na trening i validacionom skupu kod Qwen 3.5 2B modela*

Epoha	Training Loss	Validation Loss
1	0.382935	0.339133
2	0.298287	0.343254
3	0.264392	0.362977

Kod oba modela gubitak na trening skupu opada kroz epohe, što je očekivano. Gubitak na validacionom skupu se ponaša drugačije – kod *Gemma* modela opada zajedno sa trening gubitkom, dok kod *Qwen* modela posle prve epohe počinje blago da raste uprkos daljem padu trening gubitka. Ovo je indikator blagog *overfitting*-a kod *Qwen* modela, gde model nakon prve

epohe počinje da pamti specifičnosti trening skupa umesto da generalizuje, što je i očekivano jer trening skup nema puno uzoraka.

Evaluacija fino podešenih modela sprovedena je direktno u *Colab* okruženju putem *transformers* biblioteke, u istoj 4-bitnoj NF4 kvantizaciji korišćenoj tokom treninga – a ne putem *Ollama* kao kod *zero-shot* i *few-shot* režima. Razlog tome je to što konverzija u GGUF format, potreban za *Ollama*, zahteva više RAM memorije nego što besplatni *Colab T4 runtime* obezbeđuje za modele ove veličine. Kvantizacija korišćena u *Ollama* (Q4_K_M) i NF4 korišćena u *Colab*–u su obe 4-bitne, sa malom praktičnom razlikom u performansama, ali ovu razliku u okruženju izvršavanja treba imati na umu prilikom poređenja vremena izvršavanja fino podešenih modela sa ostalim režimima. Dobijene predikcije preuzete su iz *Colab*-a i ubačene u glavnu svesku zajedno sa predikcijama lokalnih modela, kako bi se sve metrike računale na jedinstven način.

Korišćenje besplatnog *Colab* okruženja unosi i određena ograničenja – ograničeno trajanje sesije i ograničena GPU memorija (T4 sa 16 GB VRAM), što je bio dodatni razlog za odabir QLoRA pristupa umesto punog finog podešavanja, kao i za odabir relativno male veličine *batch*-a.

4.5. Evaluacione metrike

Metrike se računaju iz sačuvanih predikcija – JSONL fajlova u kojima je za svaku rečenicu zapisan skup očekivanih parova aspekata i sentimenta, predikcija modela, indikator uspešnog parsiranja izlaza, vreme izvršavanja i potreban memorijski prostor za pokretanje modela. Pre nego što se uopšte može govoriti o kvalitetu predikcija, potrebno je utvrditi da li je izlaz modela bio u traženom formatu jer generativni modeli, za razliku od klasičnih klasifikatora, ne garantuju da će uvek vratiti validan JSON. Udeo izlaza koji su uspešno parsirani prema definisanoj šemi predstavlja *compliance rate*. Sve metrike kvaliteta koje slede računaju se isključivo nad uspešno parsiranim izlazima, pa *compliance rate* mora uvek biti naveden uz njih – nizak *compliance rate* bi inače mogao da naizgled poboljša ostale rezultate, ako model uspeva da vrati validan format samo na jednostavnijim primerima.

Nad parsiranim izlazima, kvalitet predikcija se sagledava na tri nivoa, koji se nadograđuju jedan na drugi. Prvi nivo je ekstrakcija aspekata (engl. *aspect term extraction – ATE*), koja odgovara prvom podzadatku originalnog *SemEval-2014 Task 4* rada [23]. Preciznost, odziv i F1 mera računaju se na nivou celog test skupa: za svaku rečenicu poredi se skup predviđenih aspekata sa skupom korektnih aspekata, nakon normalizacije teksta. Aspekt koji se nalazi i u predviđenom i u korektnom skupu predstavlja tačno pozitivan slučaj (TP), aspekt koji je predviđen a ne postoji u korektnom skupu predstavlja lažno pozitivan slučaj (FP), dok aspekt koji postoji u korektnom skupu a model ga nije predvideo predstavlja lažno negativan slučaj (FN). Ove vrednosti se sabiraju preko svih rečenica test skupa, nakon čega se preciznost, odziv i F1 mera računaju na sledeći način:

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F1 = \frac{2PR}{P + R}$$

Pogrešno predviđeni aspekti (FP) smanjuju preciznost, dok propušteni aspekti (FN) smanjuju odziv.

Drugi nivo, klasifikacija sentimenta aspekta (engl. *aspect sentiment classification – ASC*), nadograđuje se na prvi i ima smisla samo nad aspektima koje je model uspeo da pravilno ekstrahuje – drugim rečima, ako je aspekt pogođen, određuje se koliko je tačan sentiment koji mu je dodeljen. Pored tačnosti, računa se i makro-F1 preko četiri klase sentimenta, što je posebno važno jer raspodela klasa nije ravnomerna pa bi tačnost sama po sebi mogla da prikaže lažno optimistične rezultate ako model potpuno ignoriše retke klase.

Treći, najstroži nivo je *end-to-end* F1, koji predstavlja proširenje ATE metrika – umesto što se poredi samo skup aspekata, poredi se skup parova (aspekt, sentiment). Par se smatra tačnim samo ako su i aspekt i njegov sentiment ispravno predviđeni, a preciznost, odziv i F1 mera računaju se na isti način kao kod ATE, samo nad skupovima parova umesto skupovima aspekata. *End-to-end* F1 mera sumira kvalitet celog ABSA procesa u jedan broj i predstavlja glavnu metriku za poređenje modela u ovom radu. Razlika između ova dva nivoa ogleda se i u implementaciji – ATE radi nad skupom aspekata, a *end-to-end* nad skupom parova, što je prikazano u Listingu 6.

Listing 6. Računanje TP/FP/FN za ATE i end-to-end nivo

```
def _aset(pairs):  return {norm_aspect(p["aspect"]) for p in pairs}
def _pset(pairs):  return {(norm_aspect(p["aspect"]), p["sentiment"]) for p in pairs}

def _prf(tp, fp, fn):
    p = tp / (tp + fp) if tp + fp else 0.0
    r = tp / (tp + fn) if tp + fn else 0.0
    return p, r, (2 * p * r / (p + r) if p + r else 0.0)

def _micro(records, keyfn):
    tp = fp = fn = 0
    for r in records:
        g, p = keyfn(r["gold"]), keyfn(r["pred"])
        tp += len(g & p); fp += len(p - g); fn += len(g - p)
    return _prf(tp, fp, fn)
```

Pored kvaliteta predikcija, evaluacija obuhvata i praktične aspekte korišćenja modela – za svaku kombinaciju modela i režima beleže se prosečno i medijana vreme izvršavanja po rečenici, kao i potrebna memorija za upotrebu modela tamo gde je merljiva. Za lokalne modele memorija odgovara veličini modela učitanog u RAM, dok za *Gemini* API ovaj podatak nije dostupan, a vreme izvršavanja uključuje i mrežnu latenciju. Kod fino podešenih modela evaluiranih u *Colab* okruženju, izmereno vreme nije direktno uporedivo sa merenjima izvedenim lokalno jer okruženje nije isto.

4.6. Rezultati i diskusija

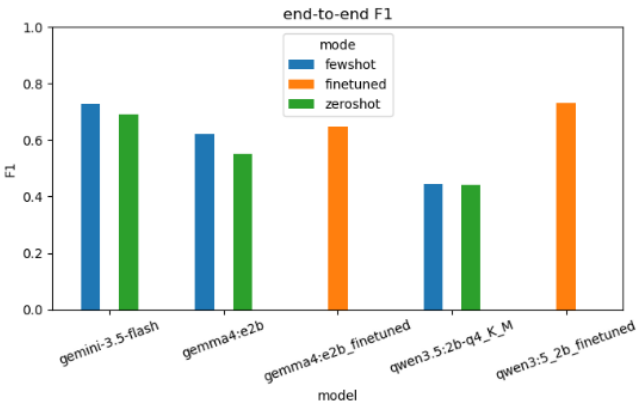
Tabela 4 prikazuje glavne rezultate za testirane kombinacije modela i režima: uspešnost parsiranja izlaza, ATE preciznost/odziv/F1, ASC tačnost i makro-F1, kao i *end-to-end* preciznost/odziv/F1. Najbolje i najgore vrednosti su naglašene u okviru svake kolone – najbolje su podebljane, a najgore podebljane i podvučene.

Tabela 4. *Glavni rezultati evaluacije*

Model	Režim	Compl.	ATE P	ATE R	ATE F1	ASC Acc	ASC mF1	E2E P	E2E R	E2E F1
Gemini 3.5 Flash	*few-shot*	**1.000**	0.805	**0.862**	0.832	0.873	**0.733**	0.706	**0.752**	0.728
Gemini 3.5 Flash	*zero-shot*	0.995	0.804	0.759	0.781	**0.885**	0.725	0.716	0.671	0.693
Gemma 4 E2B	*few-shot*	**1.000**	0.819	0.642	0.720	0.863	0.677	0.711	0.554	0.623
Gemma 4 E2B	*zero-shot*	**1.000**	0.704	0.620	0.659	0.832	0.662	0.590	0.516	0.551
Gemma 4 E2B	*fine-tuned*	0.993	0.803	0.768	0.785	0.826	0.663	0.665	0.633	0.649
Qwen 3.5 2B	*few-shot*	**1.000**	0.729	0.613	0.666	**0.663**	**0.485**	0.488	**0.408**	0.444
Qwen 3.5 2B	*zero-shot*	**0.879**	**0.614**	**0.545**	**0.577**	0.760	0.528	**0.468**	0.415	**0.440**
Qwen 3.5 2B	*fine-tuned*	**1.000**	**0.861**	0.857	**0.859**	0.849	0.689	**0.734**	0.728	**0.731**

Najupadljiviji rezultat odmah se uočava u poslednjoj koloni – fino podešen *Qwen 3.5 2B* postiže *end-to-end* F1 od 0.731, neznatno nadmašujući *Gemini 3.5 Flash* u *few-shot* režimu (0.728), koji je do tada bio najbolji rezultat u tabeli. Drugim rečima, model sa svega 2 milijarde parametara, pokrenut lokalno u 4-bitnoj kvantizaciji, dostiže nivo performansi velikog komercijalnog modela na ovom zadatku – pod uslovom da je prethodno fino podešen na nekoliko hiljada primera. Takođe, može se primetiti da fino podešavanje najviše doprinosi *Qwen* modelu. Ovaj model u *zero-shot* i *few-shot* režimima postiže najslabije rezultate u poređenju sa ostalim modelima, dok kada je fino podešen, kroz većinu metrika postiže najbolje

rezultate. Slika 4.2 ovu razliku prikazuje vizuelno, gde fino podešen *Qwen* vidno odskače od svojih *zero-shot* i *few-shot* rezultata, dostižući nivo *Gemini few-shot* konfiguracije.



Slika 4.2. End-to-end F1 grafik po modelu/režimu

Efekat finog podešavanja nije bio isti za oba SLM modela. Kod *Qwen*-a, fino podešavanje dovodi do drastičnog poboljšanja – *end-to-end* F1 raste sa 0.440 (*zero-shot*) odnosno 0.444 (*few-shot*) na 0.731, što je skoro duplo bolje. Kod *Gemma* modela je efekat znatno blaži: fino podešen model (0.649) tek neznatno nadmašuje svoj *few-shot* rezultat (0.623). Ova razlika ima smisla ako se uzmu početne performanse oba modela – *Gemma* model je već u *few-shot* režimu bio relativno sposoban za ovaj zadatak, dok je *Qwen* u *zero-shot* režimu imao i slabije ATE i ASC rezultate i, što je posebno bitno, uspešnost parsiranja rezultata od samo 0.879. Drugim rečima, *Qwen* je imao znatno više prostora za poboljšanje, pa je fino podešavanje tu donelo najveći efekat.

Upravo ovaj stepen uspešnosti parsiranja od 0.879 za *Qwen* u *zero-shot* režimu izdvaja se kao jedini slučaj u tabeli gde model nije gotovo uvek vraćao izlaz u traženom formatu – u oko 12% slučajeva izlaz nije bio validan. Već dodavanje nekoliko primera u *few-shot* upitu rešava ovaj problem, kao i fino podešavanje. Ovo ukazuje da mali model bez ikakvih primera ima poteškoća čak i sa generisanjem validnog JSON formata, ne samo sa određivanjem aspekata i sentimenata.

Kada je reč o efektu *few-shot* režima, rezultati su konzistentni – kod sva tri modela *few-shot* poboljšava *end-to-end* F1 u odnosu na *zero-shot*, mada u različitoj meri. Kod *Gemma* modela je poboljšanje najizraženije (sa 0.551 na 0.623), dok je kod *Qwen*-a (sa 0.440 na 0.444) i *Gemini*-ja (sa 0.693 na 0.728) manje izraženo. Kod *Qwen*-a je vredno napomenuti da ATE F1 blago raste (sa 0.577 na 0.666), ali ASC tačnost opada (sa 0.760 na 0.663), pa se ova dva efekta delimično poništavaju u *end-to-end* F1 metrici. Problem *Qwen* modela sa ispravnim klasifikovanjem izdvojenih aspekata u *few-shot* režimu se može videti i u Tabeli 5, gde ovaj model postiže najgore rezultate za sve četiri klase.

Posmatrajući rezultate po klasama sentimenta (Tabela 5), klasa *conflict* je dosledno najslabija kod svih modela i režima, sa F1 vrednostima u rasponu od 0.118 (*Qwen few-shot*) do 0.444 (*Gemini zero-shot*). Ovo je direktna posledica raspodele klasa u skupu podataka – *conflict* čini samo 2.5% aspekata, pa nijedan model nema dovoljno primera, ni u trening skupu ni u *few-shot* upitu, da nauči ovu klasu na zadovoljavajućem nivou.

Tabela 5. *ASC F1 po klasama sentimenta*

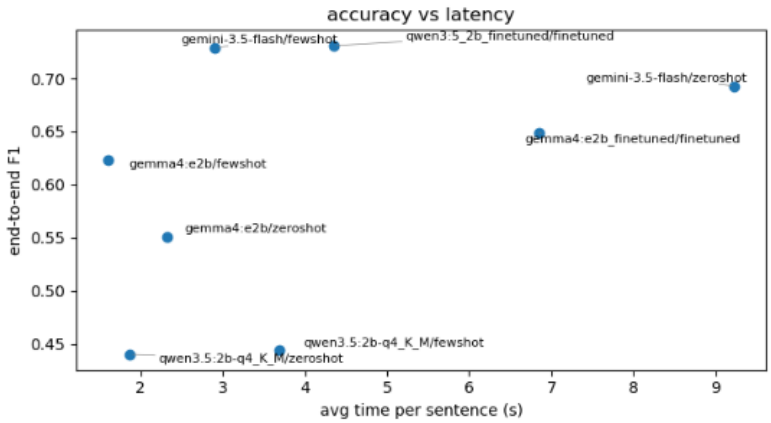
Model	Režim	*Positive*	*Negative*	*Neutral*	*Conflict*
Gemini 3.5 Flash	*few-shot*	0.941	0.847	**0.716**	0.429
Gemini 3.5 Flash	*zero-shot*	**0.948**	**0.869**	0.640	**0.444**
Gemma 4 E2B	*few-shot*	0.932	0.842	0.518	0.417
Gemma 4 E2B	*zero-shot*	0.918	0.773	0.633	0.323
Gemma 4 E2B	*fine-tuned*	0.910	0.808	0.557	0.378
Qwen 3.5 2B	*few-shot*	**0.775**	**0.701**	**0.348**	**0.118**
Qwen 3.5 2B	*zero-shot*	0.844	0.784	0.350	0.133
Qwen 3.5 2B	*fine-tuned*	0.933	0.797	0.682	0.345

Pored kvaliteta predikcija, korisno je sagledati i praktične aspekte korišćenja modela – vreme izvršavanja i potreban memorijski prostor, prikazane u Tabeli 6.

Tabela 6. *Vreme izvršavanja i potreban memorijski prostor*

Model	Režim	Pros. vreme (s)	Med. vreme (s)	Memorija (MB)
Gemini 3.5 Flash	*few-shot*	2.908	**0.903**	–
Gemini 3.5 Flash	*zero-shot*	**9.229**	1.107	–
Gemma 4 E2B	*few-shot*	**1.601**	1.354	7678.2
Gemma 4 E2B	*zero-shot*	2.318	2.069	7678.2
Gemma 4 E2B	*fine-tuned*	6.850	**5.022**	–
Qwen 3.5 2B	*few-shot*	3.690	3.554	1630.2
Qwen 3.5 2B	*zero-shot*	1.867	1.770	1630.2
Qwen 3.5 2B	*fine-tuned*	4.351	4.024	–

Razlika između prosečnog i medijalnog vremena kod *Gemini* konfiguracija je upadljiva, što ukazuje na prisustvo nekoliko ekstremnih odstupanja koja značajno povećavaju prosek bez uticaja na medijanu. Ova odstupanja se javljaju usled povremenih ponovnih pokušaja zbog mrežnih grešaka ili zbog velike zauzetosti modela u određenom trenutku. Kod lokalnih modela ova razlika je znatno manja, što je u skladu sa stabilnijim, predvidljivijim izvršavanjem na sopstvenom hardveru.



Slika 4.3. Odnos između prosečnog vremena izvršavanja po rečenici i end-to-end F1 mere za različite kombinacije modela i režima

Slika 4.3 prikazuje odnos između prosečnog vremena izvršavanja po rečenici i *end-to-end* F1 mere za svaku konfiguraciju. Generalno se uočava da lokalni SLM modeli u *zero-shot* i *few-shot* režimu spadaju u najbrže konfiguracije (1.6-3.7 s po rečenici), ali sa nižim F1 vrednostima, dok Gemini u *zero-shot* režimu predstavlja suprotan ekstrem – najsporiju konfiguraciju (9.2 s) koja pri tome nije i najtačnija. Fino podešeni modeli, evaluirani u *Colab* okruženju putem *transformers* biblioteke umesto *Ollama*, zauzimaju srednje vrednosti vremena (4.4-6.9 s), ali ova vremena nisu direktno uporediva sa lokalnim merenjima zbog razlika u okruženju izvršavanja. Najbolji odnos tačnosti i brzine, uzimajući ovu napomenu u obzir, postiže fino podešeni *Qwen* model.

Kada je reč o memoriji, *Gemma 4 E2B* pokrenuta putem *Ollama* zauzima oko 7.5 GB RAM-a u oba lokalna režima, dok *Qwen 3.5 2B* zauzima oko 1.6 GB, što je razlika koja grubo odgovara razlici u veličini ova dva modela u Q4_K_M kvantizaciji. Za *Gemini* API memorija nije merljiva jer se izvršavanje odvija na eksternoj infrastrukturi, a za fino podešene modele ovaj podatak nije zabeležen u rezultatima *Colab* evaluacije.

Konačno, vredi istaći nekoliko ograničenja ovog eksperimenta. Test skup od 405 rečenica relativno je mali, naročito za retke klase poput *conflict*, gde svaka pogrešna predikcija ima veliki relativni uticaj na F1. *Few-shot* upit je sadržao samo četiri primera – po jedan za svaku klasu sentimenta – pa veći broj ili drugačiji odabir primera potencijalno bi mogao dati drugačije rezultate. Konačno, razlike u kvantizaciji i okruženju izvršavanja između *Ollama* (Q4_K_M, CPU) i *Colab* (NF4, GPU) za fino podešene modele unose dodatnu nekonzistentnost u poređenju vremena.

5. ZAKLJUČAK

U ovom radu istraženo je u kojoj meri mali i veliki jezički modeli mogu da reše zadatak analize sentimenta na nivou aspekata, korišćenjem *SemEval-2014 Task 4* skupa podataka restoranskih recenzija. Poređeni su dva mala, lokalno pokrenuta modela – *Gemma 4 E2B* i *Qwen 3.5 2B* – i jedan veliki komercijalni model dostupan putem API-ja, *Gemini 3.5 Flash*, u tri eksperimentalna režima: *zero-shot*, *few-shot* i *fine-tuning*.

Glavni rezultat rada je da fino podešen *Qwen 3.5 2B*, model sa svega 2 milijarde parametara pokrenut u 4-bitnoj kvantizaciji, dostiže *end-to-end* F1 od 0.731 – neznatno iznad rezultata *Gemini 3.5 Flash* u *few-shot* režimu (0.728), koji je do tada bio najbolji rezultat među svim konfiguracijama. Efekat finog podešavanja pri tom je zavisio od inicijalnog ponašanja modela – kod *Qwen*-a, koji je u *zero-shot* režimu imao slabije rezultate i nizak stepen generisanja validnog JSON formata, fino podešavanje je donelo gotovo dvostruko poboljšanje, dok je kod *Gemma* modela, koji je već u *few-shot* režimu bio solidan, efekat bio znatno blaži. *Few-shot* primeri su kod sva tri modela doveli do poboljšanja u odnosu na *zero-shot*, a klasa *conflict* ostala je najslabije prepoznata kod svih modela, što je direktna posledica njene male zastupljenosti u skupu podataka.

Rezultati ukazuju da mali jezički modeli, uz odgovarajuće fino podešavanje, mogu da pariraju znatno većim i računarski zahtevnijim modelima na domenski specifičnim zadacima poput ABSA. Dalja istraživanja mogla bi obuhvatiti evaluaciju na većem test skupu sa više pokretanja po konfiguraciji, ispitivanje uticaja broja i izbora *few-shot* primera, kao i proširenje eksperimenata na druge domene ili druge male i velike jezičke modele.

LITERATURA

[1] "What is sentiment analysis?," IBM, [Online]. Available: https://www.ibm.com/think/topics/sentiment-analysis. [Accessed 9 June 2026].

[2] "What is sentiment analysis?," elastic, [Online]. Available: https://www.elastic.co/what-is/sentiment-analysis. [Accessed 9 June 2026].

[3] F. Chiusano, "Quick intro to Aspect-Based Sentiment Analysis," Medium, 19 August 2022. [Online]. Available: https://medium.com/nlplanet/quick-intro-to-aspect-based-sentiment-analysis-c8888a09eda7. [Accessed 9 June 2026].

[4] P. Huoviala and P. F. Simmering, "Large Language Models for aspect-based sentiment analysis," arXiv, 2023.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser and I. Polosukhin, "Attention Is All You Need," in *31st Conference on Neural Information Processing Systems (NIPS 2017)*, Long Beach, CA, USA, 2017.

[6] C. Stryker and D. Bergmann, "What is a transformer model?," IBM, [Online]. Available: https://www.ibm.com/think/topics/transformer-model. [Accessed 10 June 2026].

[7] "What is Pre Training and its Objective," GeeksForGeeks, 9 May 2026. [Online]. Available: https://www.geeksforgeeks.org/artificial-intelligence/what-is-pre-training-and-its-objective/. [Accessed 10 June 2026].

[8] "An Ultimate Guide To Transfer Learning In NLP," 5 December 2019. [Online]. Available: https://www.topbots.com/transfer-learning-in-nlp/. [Accessed 10 June 2026].

[9] "Zero-Shot Learning vs. Few-Shot Learning vs. Fine-Tuning: A technical walkthrough using OpenAI's APIs & models," Labelbox, [Online]. Available: https://labelbox.com/guides/zero-shot-learning-few-shot-learning-fine-tuning/. [Accessed 11 June 2026].

[10] S. Sacatani, "Zero-Shot Learning vs. Few-Shot Learning vs. Fine Tuning," Medium, 2 September 2024. [Online]. Available: https://medium.com/@sophiamsac/zero-shot-learning-vs-few-shot-learning-vs-fine-tuning-07e6c9fd4fda. [Accessed 11 June 2026].

[11] D. Sharma, "Fine-Tuning LLMs with LoRA and QLoRA: From Confusion to (Kinda) Working Results," Medium, 5 July 2025. [Online]. Available: https://medium.com/@dsh.2065/fine-tuning-llms-with-lora-and-qlora-from-confusion-to-kinda-working-results-89b348bcce71. [Accessed 11 June 2026].

[12] M. Kondah, "Parameter-Efficient Fine-Tuning (PEFT), LoRA and Quantization," 8 October 2023. [Online]. Available: https://www.deep-kondah.com/parameter-efficient-fine-tuning-peft-lora-and-quantization/. [Accessed 11 June 2026].

[13] J. Johnson, "Small Language Models (SLM): A Comprehensive Overview," HuggingFace, 22 February 2025. [Online]. Available: https://huggingface.co/blog/jjokah/small-language-model. [Accessed 12 June 2026].

[14] S. Cole, "What are large language models (LLMs)?," IBM, [Online]. Available: https://www.ibm.com/think/topics/large-language-models. [Accessed 12 June 2026].

[15] K. Naminas, "LLM Model Size: Comparison Chart & Performance Guide," Label Your Data, 19 December 2024. [Online]. Available: https://labelyourdata.com/articles/llm-fine-tuning/llm-model-size. [Accessed 12 June 2026].

[16] "SLMs vs LLMs: What are small language models?," RedHat, 12 May 2026. [Online]. Available: https://www.redhat.com/en/topics/ai/llm-vs-slm. [Accessed 12 June 2026].

[17] C. S. Rina Diane Caballar, "What are small language models?," IBM, [Online]. Available: https://www.ibm.com/think/topics/small-language-models. [Accessed 12 June 2026].

[18] T. Lu, "SLMs vs LLMs: A Complete Guide to Small Language Models and Large Language Models," DataCamp, 30 September 2025. [Online]. Available: https://www.datacamp.com/blog/slms-vs-llms. [Accessed 12 June 2026].

[19] HuggingFace, [Online]. Available: https://huggingface.co/google/gemma-4-E2B. [Accessed 13 June 2026].

[20] HuggingFace, [Online]. Available: https://huggingface.co/Qwen/Qwen3.5-2B. [Accessed 13 June 2026].

[21] K. Kavukcuoglu, J. Dean, O. Vinyals and N. Shazeer, "Gemini 3.5: frontier intelligence with action," Google, 19 May 2026. [Online]. Available: https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-5/#gemini-3-5-flash. [Accessed 13 June 2026].

[22] "SemEval 2014 Task 4: AspectBasedSentimentAnalysis," Kaggle, [Online]. Available: https://www.kaggle.com/datasets/charitarth/semeval-2014-task-4-aspectbasedsentimentanalysis?select=Restaurants_Train_v2.csv.

[23] M. Pontiki, H. Papageorgiou, D. Galanis, I. Androutsopoulos, J. Pavlopoulos and S. Manandhar, "SemEval-2014 Task 4: Aspect Based Sentiment Analysis," in *Proceedings of the 8th International Workshop on Semantic Evaluation (SemEval 2014)*, Dublin, Ireland, 2014.